

PolynomiaZ: Polar Coordinate Compression via Mathematical Function Fitting on Virtual Disk Platters

PF Works

May 2026

Abstract

We present PolynomiaZ (PLTZ), a lossless compression codec that maps input data onto a virtual disk platter geometry and detects mathematical patterns within each track. Tracks matching a known pattern type are stored as compact function descriptors rather than raw bytes. PLTZ achieves 5–12× better compression than zlib on linearly structured numerical data (firmware images, satellite telemetry, SPICE ephemeris kernels) while maintaining less than 1% overhead on incompressible data. The codec supports adaptive sector sizing, multi-platter splitting, configurable raw fallback compression, and word-width-aware analysis for 16/32/64-bit numerical arrays.

1 Introduction

Standard compression algorithms (zlib, bzip2, LZMA) operate by identifying byte-level repetition through sliding windows, Burrows-Wheeler transforms, or dictionary coding. These approaches excel at data with repeated substrings—natural language text, markup, log files—but fundamentally cannot recognize that a byte sequence like $[0, 1, 2, 3, \dots, 255]$ represents a linear function. They see “no repeated substrings” and store it nearly verbatim.

PolynomiaZ takes a different approach: it treats the input as data written to a virtual disk platter and asks, for each track, “what mathematical function produced these bytes?” When a function is found, the track is stored as a compact descriptor (e.g., `LINEAR(start=0, step=1)` in 4 bytes instead of 256 raw bytes).

This approach is not general-purpose. It targets a specific domain: data with known mathematical structure arising from physical processes, calibration sequences, address tables, and numerical computation.

2 Architecture

2.1 Disk Platter Model

Input data of length N bytes is mapped onto a virtual disk with geometry (T, S) where T is the number of concentric tracks and S is the number of sectors per track, such that $T \times S \geq N$. Each byte occupies a position (r, θ) in polar coordinates:

$$r = \lfloor i/S \rfloor \quad (\text{track index}) \tag{1}$$

$$\theta = \frac{2\pi(i \bmod S)}{S} \quad (\text{angular position}) \tag{2}$$

where i is the byte’s sequential position in the input.

2.2 Adaptive Geometry Selection

The codec evaluates multiple sector sizes $S \in \{8, 16, 32, 64, 128, 256, 512\}$ and selects the geometry that produces the smallest compressed output. Different data benefits from different geometries: small sectors expose patterns in structured headers, while large sectors capture long arithmetic sequences.

2.3 Multi-Platter Splitting

After analysis, tracks are separated into two platters:

- **Platter 0:** Tracks with detected patterns (structured data)
- **Platter 1:** Tracks with no pattern (raw data, optionally compressed with a fallback algorithm)

This separation allows the raw fallback compressor to operate on contiguous unstructured data without interference from the structured tracks.

3 Pattern Detection

Each track is tested against pattern detectors in priority order. The first match wins (or the smallest encoding is chosen when multiple patterns match).

3.1 Pattern Types

Tag	Pattern	Detection Criterion	Storage Cost
0	CONST	All bytes identical	1 byte
1	LINEAR	$x_i = a + bi$ for all i	4 bytes
2	RLE	Run-length encoding saves space	$2n_{\text{runs}}$ bytes
3	REPEAT	Repeating sub-pattern of period $p < S/3$	p bytes
4	DELTA	≤ 8 unique deltas, all in $[-128, 127]$	$S + 1$ bytes
5	PERIODIC	FFT with k significant components	$4 + 18k$ bytes
6	POLY	Polynomial degree ≤ 8 exact fit	$1 + 8(d + 1)$ bytes
9	LINEAR_WIDE	Linear sequence in 16/32-bit words	19 bytes
10	LINEAR_F64	Linear sequence in IEEE 754 doubles	18 bytes
11	DELTA_F64	f64 values with f32-representable deltas	$10 + 4(n - 1)$ bytes
7	RAW	No pattern (fallback)	S bytes
8	RAW_COMPRESSED	RAW with deflate applied	$3 + n_c$ bytes

Table 1: Pattern types and their storage costs. S = sectors per track, d = polynomial degree, k = FFT components, n_c = compressed size.

3.2 Wide-Word Analysis

A key innovation is reinterpreting byte tracks as arrays of wider integer or floating-point types. A track of 128 bytes that appears random at the byte level may contain 32 linearly-incrementing 32-bit addresses:

[0x80000000, 0x80000020, 0x80000040, ...]

The LINEAR_WIDE detector unpacks bytes as little-endian 16-bit or 32-bit unsigned integers and checks for arithmetic sequences. LINEAR_F64 and DELTA_F64 do the same for IEEE 754 double-precision values, with byte-exact reconstruction verification.

3.3 Lossless Guarantee

All pattern detectors verify byte-exact reconstruction before accepting a match. For floating-point patterns (LINEAR_F64, DELTA_F64), the reconstructed bytes are compared against the original—not just the numerical values—ensuring no precision loss from floating-point arithmetic.

4 Binary Format

```
Header (12 bytes):
[4] Magic "PLTZ"
[4] Original data length (uint32 LE)
[2] Sectors per track (uint16 LE)
[2] Number of platters (uint16 LE)

Per platter:
[2] Number of tracks (uint16 LE)
Per track:
[2] Track index (uint16 LE)
[1] Pattern tag (uint8)
[...] Pattern-specific payload
```

The format has a fixed 12-byte header overhead. Each track costs 3 bytes for its index and tag, plus the pattern-specific payload. For a file with all tracks matching CONST, the overhead is $12 + 2 + T \times 4$ bytes regardless of input size.

5 Experimental Results

5.1 Structured Data (PLTZ Advantage)

Data	Original	PLTZ	zlib	Improvement
Linear ramp (byte)	4,096 B	126 B	315 B	$2.5\times$
Linear ramp (f64 epochs)	8,192 B	350 B	1,881 B	$5.4\times$
Vector table (u32)	1,024 B	58 B	88 B	$1.5\times$
Segment addresses (u32)	4,096 B	182 B	2,244 B	$12.3\times$
Firmware (padding + headers)	4,096 B	90 B	307 B	$3.4\times$
Mixed structured + random	4,096 B	2,150 B	2,381 B	$1.1\times$

Table 2: PLTZ vs. zlib on mathematically structured data.

5.2 Unstructured Data (PLTZ Disadvantage)

Data	Original	PLTZ	zlib	Reason
English text	1,580 B	1,656 B	680 B	Statistical, not mathematical
Random data	4,096 B	4,134 B	4,109 B	Incompressible; 1% overhead
Constant fill 64KB	65,536 B	526 B	84 B	zlib RLE more byte-efficient

Table 3: Cases where standard compressors outperform PLTZ.

5.3 Domain-Specific Benchmarks

5.3.1 U-Boot Firmware

Per-section analysis of a synthetic 512KB U-Boot image shows PLTZ winning on structured sections (vector tables: 36B vs. 88B, relocation tables: 102B vs. 576B, PLL config: 36B vs. 201B) while losing on machine code and large constant fills.

5.3.2 SPICE Ephemeris Kernels

For JPL SPICE kernel structures, PLTZ achieves $5.4\times$ better compression than zlib on linearly-spaced epoch arrays (the time index present in every SPK/CK file) and $12.3\times$ on segment address directories. Chebyshev polynomial coefficients (the bulk of SPK data) remain better served by LZMA due to their high-entropy nature.

6 Target Applications

PLTZ is designed for domains where data originates from mathematical processes:

1. **Embedded firmware:** Flash images with 0xFF padding, address tables, calibration ramps
2. **Satellite telemetry:** Housekeeping data, attitude quaternions, epoch arrays, CCSDS framing
3. **Scientific instruments:** ADC/DAC sweeps, spectral calibration, sensor linearization tables
4. **SPICE kernels:** Epoch directories, segment address tables, orientation polynomials
5. **Protocol framing:** Sync patterns, incrementing sequence numbers, structured headers
6. **Lookup tables:** Gamma curves, PWM tables, motor control profiles

7 Implementation

The reference implementation is written in Rust with a bzip2-style command-line interface:

```
pltz file.bin          # compress -> file.bin.pltz
pltz -d file.bin.pltz  # decompress -> file.bin
pltz -k file.bin       # keep original
pltz -z file.bin       # enable zlib fallback for raw tracks
pltz -c file.bin       # output to stdout (pipeable)
```

The binary is approximately 1MB stripped, with no runtime dependencies beyond libc. Compression speed is dominated by the adaptive geometry search ($7 \text{ sector sizes} \times \text{pattern detection per track}$).

8 Comparison with Existing Work

PLTZ occupies a unique niche: it recognizes *semantic* structure (“this is a linear function”) rather than *syntactic* structure (“these bytes appeared before”). The two approaches are complementary—PLTZ with a zlib fallback combines the strengths of both.

Algorithm	Strategy	Best Domain
zlib (DEFLATE)	LZ77 + Huffman	Repeated substrings
bzip2	BWT + RLE + Huffman	Long-range repetition
LZMA	Large dictionary + range coding	General purpose
PLTZ	Function fitting on polar geometry	Mathematical structure

Table 4: Algorithmic comparison.

9 Limitations and Future Work

Current limitations:

- Single geometry for the entire file (mixed-structure files need pre-segmentation)
- No cross-track pattern detection (same function on multiple tracks stored redundantly)
- Periodic detection requires exact FFT reconstruction (integer rounding breaks it)
- Slower than zlib due to adaptive geometry search

Planned improvements:

- Columnar pre-processing transform for record-oriented data
- Cross-track correlation (detect identical patterns across tracks)
- Streaming/chunked mode for large files
- Per-section geometry (different sector sizes for different regions)

10 Conclusion

PolynomiaZ demonstrates that domain-specific compression based on mathematical function fitting can dramatically outperform general-purpose algorithms on structured numerical data. By modeling data as points on a virtual disk platter and fitting functions per track, PLTZ achieves 5–12 $\times$ better compression than zlib on linear sequences, address tables, and epoch arrays—exactly the data structures found in firmware, telemetry, and scientific computing. The approach is complementary to existing compressors and is most effective when used as part of a hybrid pipeline.